Table des matières

Cours 04 — Les codes 1
Étape 1 — Ce qui est une vente · INTERMEDIATE 1
Étape 2 — La règle, `int_ventes` · INTERMEDIATE 2
Étape 3 — La date, `dim_date` · MARTS 3
Étape 4 — Les clés de substitution, `dim_produit` et `dim_membre` · MARTS 3
Étape 5 — Le fait, `fct_ventes` · MARTS 4
Étape 6 — Lire l'étoile, puis la réconcilier · MARTS 4

Cours 04 — Les codes

Cours 4/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
	✓ ici	

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin, et chaque requête du notebook sous son étape ; les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule, sous celle qui attache l'entrepôt.

Étape 1 — Ce qui est une vente · INTERMEDIATE

1.2 Les tickets par type et par statut

```sql
-- 1. les tickets, par type et par statut
select type_ticket, statut, count(*) as tickets
from brulerie.stg_tickets
group by type_ticket, statut
order by type_ticket, statut;
```

1.3 Les premiers tickets annulés

© 2026 iAlexMG. Tous droits réservés.

```sql
-- 1. les cinq premiers tickets annulés : ce qu'ils valaient, ce qu'ils ont rapporté
select ticket_id, sous_total, mode_paiement, encaisse
from brulerie.stg_tickets
where statut = 'ANNULE'
order by ticket_id
limit 5;
```

1.4 Le remboursement

```sql
-- 1. le remboursement du mois : son ticket, sa date, la vente d'origine, ses montants
select ticket_id, horodatage, ticket_origine, sous_total, encaisse
from brulerie.stg_tickets
where type_ticket = 'REMBOURSEMENT';
```

Étape 2 — La règle, `int_ventes` · INTERMEDIATE

02_ae/dbt_project.yml

```yaml
# 02_ae/dbt_project.yml : le projet dbt de La Brûlerie
name: brulerie
version: "1.0.0"
# 1. le profil de connexion à lire dans profiles.yml
profile: brulerie

# 2. un dossier de models/ par couche ; le STAGING et l'INTERMEDIATE en vues, les MARTS en tables
models:
  brulerie:
    staging:
      +materialized: view
    intermediate:
      +materialized: view
    marts:
      +materialized: table
```

02_ae/models/intermediate/int_ventes.sql

```sql
-- 02_ae/models/intermediate/int_ventes.sql : les lignes des tickets payés, avec leur date et leur membre
select
    -- 1. la ligne, avec la date et le membre de son ticket
    l.ticket_id,
    l.ligne,
```


© 2026 iAlexMG. Tous droits réservés.

```
    t.horodatage::date as date_vente,
    t.membre_id,
    l.produit_id,
    l.quantite,
    l.prix_unitaire,
    l.rabais,
    l.montant
-- 2. les lignes et leur ticket, lus par ref()
from {{ ref('stg_lignes') }} as l
inner join {{ ref('stg_tickets') }} as t on t.ticket_id = l.ticket_id
-- 3. ce qui est une vente : un ticket payé ; les annulés sortent, le remboursement reste
where t.statut = 'PAYE'
```

Étape 3 — La date, `dim_date` · MARTS

02_ae/models/marts/dim_date.sql

```
-- 02_ae/models/marts/dim_date.sql : les 30 jours de juin 2025, un par ligne
select
    -- 1. la clé : la date elle-même
    jour::date as date_jour,
    -- 2. le nom du jour, du lundi (1) au dimanche (7)
    case isodow(jour)
        when 1 then 'lundi' when 2 then 'mardi' when 3 then 'mercredi'
        when 4 then 'jeudi' when 5 then 'vendredi' when 6 then 'samedi'
        when 7 then 'dimanche'
    end as nom_jour,
    -- 3. la fin de semaine : le samedi et le dimanche
    isodow(jour) >= 6 as est_fin_de_semaine
-- 4. une ligne par jour, du 1er au 30 juin, les deux bornes comprises
from generate_series(date '2025-06-01', date '2025-06-30', interval 1 day) as jours(jour)
```

Étape 4 — Les clés de substitution, `dim_produit` et `dim_membre` · MARTS

02_ae/models/marts/dim_produit.sql

```
-- 02_ae/models/marts/dim_produit.sql : un produit du catalogue par ligne
select
    -- 1. la clé de substitution, tirée du numéro du produit
    md5(produit_id::varchar) as cle_produit,
    -- 2. le numéro de la caisse, puis ce par quoi on regarde les ventes
```


© 2026 iAlexMG. Tous droits réservés.

```
    produit_id,
    nom,
    categorie,
    format
from {{ ref('stg_produits') }}
```

02_ae/models/marts/dim_membre.sql

```
-- 02_ae/models/marts/dim_membre.sql : un membre du programme de fidélité par ligne
select
    -- 1. la clé de substitution, tirée du numéro de la carte
    md5(membre_id) as cle_membre,
    -- 2. le numéro de la carte, puis ce par quoi on regarde les ventes
    membre_id,
    secteur_postal,
    inscrit_le::date as date_inscription
from {{ ref('stg_membres') }}
```

Étape 5 — Le fait, `fct_ventes` · MARTS

02_ae/models/marts/fct_ventes.sql

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait
    ticket_id,
    ligne,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
```

Étape 6 — Lire l’étoile, puis la réconcilier · MARTS

© 2026 iAlexMG. Tous droits réservés.

6.5 Le grain

```sql
-- 1. les couples (ticket, ligne) que le fait porte plus d'une fois
select ticket_id, ligne, count(*) as fois
from brulerie.fct_ventes
group by ticket_id, ligne
having count(*) > 1;
```

6.7 Le témoin

```sql
-- 1. le témoin : la somme des sous-totaux des tickets payés, tels que la caisse les a imprimés
select sum(sous_total) as caisse
from brulerie.stg_tickets
where statut = 'PAYE';
```

6.8 Le montant par catégorie

```sql
-- 1. le montant vendu par catégorie : le fait joint à la dimension des produits
select p.categorie, sum(f.montant) as montant
from brulerie.fct_ventes as f
inner join brulerie.dim_produit as p on p.cle_produit = f.cle_produit
group by p.categorie
order by p.categorie;
```


© 2026 iAlexMG. Tous droits réservés.